

CO5 – AT2

Pseudocode Writing Task (Algorithm Design)

REPORT:

Subset Sum Problem Using Backtracking

Aim

To design and implement a backtracking algorithm to determine whether a subset of given integers exists whose sum is equal to a specified target value.

Problem Statement

Given a set of integers and a target sum, the objective is to find whether any subset of the given numbers has a sum equal to the target.

For example:

Numbers = {3, 34, 4, 12, 5, 2}

Target = 9

A valid subset is:

{3, 4, 2}

because:

$$3 + 4 + 2 = 9$$

Therefore, the output is **True**.

Inputs

- A set/list of integers.
- A target sum.

Example:

Numbers = {3, 34, 4, 12, 5, 2}

Target = 9

Output

The algorithm produces:

- **True and the subset**, if a valid subset exists.

- **False / No subset found**, if no valid subset exists.

Backtracking Concept

Backtracking explores different combinations of numbers by making two choices for every number:

1. **Include** the number in the subset.
2. **Exclude** the number from the subset.

If the current sum becomes equal to the target, the required subset is found.

If the current sum exceeds the target, the branch can be stopped and the algorithm backtracks.

Algorithm

1. Start with an empty subset and current sum = 0.
2. Select the next number.
3. Include the number in the current subset.
4. Check whether the new sum equals the target.
5. If it does, return the subset.
6. If the sum exceeds the target, prune that branch.
7. If including the number does not produce a solution, remove it.
8. Try excluding the number.
9. Continue recursively until a solution is found or all possibilities are exhausted.
10. If no subset is found, return false.

Pseudocode

BACKTRACK(numbers, target, index, subset, sum)

IF sum = target

 OUTPUT subset

 RETURN TRUE

IF sum > target OR index = length(numbers)

 RETURN FALSE

```

// Include current number
ADD numbers[index] to subset
IF BACKTRACK(numbers, target, index + 1,
    subset, sum + numbers[index])
    RETURN TRUE
REMOVE numbers[index] from subset
// Exclude current number
IF BACKTRACK(numbers, target, index + 1,
    subset, sum)
    RETURN TRUE
RETURN FALSE
MAIN
    READ numbers
    READ target
    IF BACKTRACK(numbers, target, 0, empty subset, 0)
        OUTPUT "Subset found"
    ELSE
        OUTPUT "No subset found"
END

```

Branching

For every number, the algorithm creates two branches:

Number	
/	\
Include	Exclude
/	\
Next	Next

For example, for {3, 4}:

```

      Start
    /   \
  +3    -3
 / \   / \
+4 -4 +4 -4

```

This allows the algorithm to examine different subsets.

Pruning Conditions

Pruning avoids unnecessary computations.

Condition 1: Target Found

If:

```
current_sum == target
```

the algorithm immediately stops because a valid subset has been found.

Condition 2: Sum Exceeds Target

If all numbers are non-negative and:

```
current_sum > target
```

the current branch cannot produce the target, so it is stopped.

Condition 3: All Elements Processed

If:

```
index == length(numbers)
```

there are no more numbers to examine, so the branch ends.

Example

Consider:

Numbers = {3, 34, 4, 12, 5, 2}

Target = 9

The algorithm explores different combinations.

One successful path is:

3 → Include

4 → Include

2 → Include

Therefore:

$$3 + 4 + 2 = 9$$

Hence:

Subset = {3, 4, 2}

Result = True

Time Complexity

For n numbers, every number has two choices:

- Include
- Exclude

Therefore, the worst-case number of possibilities is:

$$2^n$$

Hence, the worst-case time complexity is:

$$O(2^n)$$

Pruning can reduce the actual number of branches explored, but the worst-case complexity remains exponential.

Space Complexity

The recursive algorithm stores:

- The current subset.
- The recursion stack.
- The current index and sum.

The maximum recursion depth is n.

Therefore, the auxiliary space complexity is:

$$O(n)$$

The subset can also contain at most n elements, so the overall auxiliary space remains:

Applications

The Subset Sum Problem has applications in:

- Resource allocation.
- Cryptography.
- Financial planning.
- Combinatorial optimization.
- Partitioning problems.
- Scheduling and decision-making systems.

Result

The Subset Sum Problem was successfully implemented using the backtracking technique. The algorithm explores both inclusion and exclusion choices and uses pruning to eliminate branches that cannot produce the target sum.

For the sample input:

Numbers = {3, 34, 4, 12, 5, 2}

Target = 9

the algorithm successfully finds:

{3, 4, 2}